

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas – ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático II - Ordenação Externa

BCC203 - Estruturas de Dados II

Rafael Prado Mol e Silva (25.1.4002)
Pedro Lucca de Castro (25.1.4005)
Mateus Oliveira Heleno (25.1.4007)
João Felipe Paiva Paixão (25.1.4014)
Pedro Henrique Menezes Feo de Castro (25.1.4017)

Professor: Guilherme Tavares de Assis

Ouro Preto
23 de junho de 2026

1 Introdução

Este trabalho prático apresenta um estudo comparativo profundo sobre a complexidade de desempenho de três algoritmos de ordenação externa:

1. Intercalação Balanceada de Vários Caminhos (2f fitas) com ordenação interna na geração de blocos.
2. Intercalação Balanceada de Vários Caminhos (2f fitas) utilizando a técnica de Seleção por Substituição na geração de blocos.
3. Quicksort Externo.

O objetivo central consiste em analisar a eficiência dessas abordagens ao manipularem grandes volumes de dados que superam as limitações de capacidade da memória RAM, tornando essencial o gerenciamento inteligente de acessos à memória externa. Para a validação experimental, os métodos foram implementados na linguagem C e submetidos a testes com diferentes volumetrias de registros. O desempenho de cada algoritmo foi rigorosamente mensurado com base no número de leituras e escritas em disco, total de comparações de chaves e tempo total de execução, considerando cenários de ordenação inicial ascendente, descendente e aleatória.

1.1 Descrição dos Testes

Para avaliar e comparar a eficiência e a escalabilidade de cada método de ordenação externa, foi planejado e executado um mapeamento experimental estruturado em diferentes cenários de estresse e ordenação de dados. Os testes foram divididos com base nos seguintes critérios:

- **Quantidade de Registros:** Para analisar o comportamento dos algoritmos à medida que o volume de dados em memória externa cresce, foram processadas cinco ordens de grandeza distintas extraídas da base de dados real: 100, 1.000, 10.000, 100.000 e 471.705 registros. Cada registro possui tamanho fixo composto pelo número de inscrição do aluno (long int), nota (float), estado (cadeia de 2 caracteres), cidade (cadeia de 50 caracteres) e curso (cadeia de 30 caracteres).
- **Cenários de Ordenação Inicial:** Para cada tamanho de arquivo, foram testadas três situações distintas de organização inicial das chaves na entrada, simulando diferentes condições de uso real: arquivos ordenados ascendentemente pelas notas (Situação 1), arquivos ordenados descendentemente pelas notas (Situação 2) e arquivos originais desordenados aleatoriamente (Situação 3).
- **Restrições de Memória e Recursos:** Em conformidade com as diretrizes do projeto, os experimentos foram executados limitando estritamente a memória interna disponível a um vetor de, no máximo, 20 registros para todos os métodos. Adicionalmente, para os métodos 1 e 2, foi considerada a existência de 40 fitas de armazenamento externo simuladas em disco, divididas simetricamente entre 20 fitas de entrada e 20 fitas de saída a cada etapa de intercalação.

- **Métricas de Desempenho (Quesitos de Análise):** Os testes monitoraram e registraram os dados de desempenho computando o número de transferências de leitura de registros, o número de transferências de escrita de registros, o total de comparações efetuadas entre as notas dos alunos e o tempo de execução (calculado pela diferença exata entre o término e o início do processo de ordenação).

1.2 Instruções de Execução

Para executar o projeto, deve-se utilizar a seguinte linha de comando no console:

1. Execução do projeto:

```
1 ordena <metodo> <quantidade> <situacao> [-P]
2
```

2. **<metodo>**: representa o método de ordenação externa a ser executado, podendo ser um número inteiro de 1 a 3, de acordo com a ordem dos métodos mencionados
3. **<quantidade>**: representa a quantidade de alunos (linhas do arquivo texto) a serem ordenados.
4. **<situacao>**: representa a situação de ordem inicial do arquivo, podendo ser 1 (arquivo ordenado ascendentemente pelas notas), 2 (arquivo ordenado descendentemente pelas notas) ou 3 (arquivo desordenado aleatoriamente pelas notas).
5. **[-P]**: representa um argumento opcional que deve ser colocado quando se deseja que os dados dos alunos a serem ordenados e o resultado da ordenação realizada sejam apresentados na tela.

2 Análise Experimental de Desempenho

Para cada teste feito nos métodos abaixo, foi feito a média após 3 execuções do programa.

2.1 Intercalação Balanceada de Vários Caminhos com ordenação interna na geração de blocos

A Intercalação Balanceada de Vários Caminhos com ordenação interna na geração de blocos é um método de ordenação externa projetado para arquivos que superam a capacidade da memória RAM. O método funciona em duas fases, primeiro o algoritmo lê o arquivo original em blocos de até 20 registros, ordena-os na memória interna através do *Quicksort Interno* e os distribui sequencialmente entre 20 fitas magnéticas virtuais de entrada. Na fase seguinte, o código utiliza um *Heap-minímo* em RAM para ler simultaneamente o primeiro elemento de cada fita, gravando o menor deles em uma das 20 fitas de saída e substituindo-o imediatamente pelo próximo registro de sua fita de origem. Esse processo de fusão se repete alternando o papel das fitas a cada passada até que todos os blocos sejam unificados em um único arquivo final totalmente ordenado.

2.1.1 Testes com 100 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 200 | Escrita: 200 | Comparações: 542 | Tempo total: 0,001857 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 200 | Escrita: 200 | Comparações: 570 | Tempo total: 0,001936 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 200 | Escrita: 200 | Comparações: 877 | Tempo total: 0,002172 ms.

2.1.2 Testes com 1.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 3.000 | Escrita: 3.000 | Comparações: 6.861 | Tempo total: 0,006393 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 3.000 | Escrita: 3.000 | Comparações: 6.932 | Tempo total: 0,005880 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 3.000 | Escrita: 3.000 | Comparações: 12.654 | Tempo total: 0,007608 ms.

2.1.3 Testes com 10.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 40.000 | Escrita: 40.000 | Comparações: 84.544 | Tempo total: 0,034963 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 40.000 | Escrita: 40.000 | Comparações: 80.382 | Tempo total: 0,030363 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 40.000 | Escrita: 40.000 | Comparações: 152.298 | Tempo total: 0,035185 ms.

2.1.4 Testes com 100.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 400.000 | Escrita: 400.000 | Comparações: 960.839 | Tempo total: 0,151660 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 400.000 | Escrita: 400.000 | Comparações: 958.074 | Tempo total: 0,142381 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 400.000 | Escrita: 400.000 | Comparações: 1.687.249 | Tempo total: 0,167314 ms.

2.1.5 Testes com 471.705 de Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 2.358.525 | Escrita: 2.358.525 | Comparações: 5.024.565 | Tempo total: 0,686667 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 2.358.525 | Escrita: 2.358.525 | Comparações: 5.011.679 | Tempo total: 0,694730 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 2.358.525 | Escrita: 2.358.525 | Comparações: 8.509.828 | Tempo total: 0,796439 ms.

2.2 Intercalação Balanceada de Vários Caminhos utilizando a de Seleção por Substituição na geração de blocos

A Intercalação Balanceada de Vários Caminhos utilizando a técnica de Seleção por Substituição na geração de blocos é um método de ordenação externa focado em reduzir as passadas no disco através da criação de blocos iniciais maiores. O método funciona em duas fases, na primeira o algoritmo preenche um *Heap-minímo* em RAM com até 20 registros e grava o menor elemento na fita de saída atual. Na sequência, um novo registro é lido do arquivo original; se sua nota for maior ou igual à do que acabou de ser gravado, ele entra no *Heap* normalmente, mas se for menor, ele é marcado para o próximo bloco e fica "congelado". Quando todos os elementos do *Heap* são marcados, o bloco atual é fechado, a próxima fita de saída é selecionada, os registros são desmarcados e o processo recomeça, gerando blocos que têm, em média, o dobro do tamanho da memória interna. A segunda fase realiza a intercalação multidirecional dessas fitas de maneira idêntica ao método tradicional, fundindo os blocos até unificar todo o arquivo.

2.2.1 Testes com 100 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 100 | Escrita: 100 | Comparações: 294 | Tempo total: 0,000674 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 200 | Escrita: 200 | Comparações: 879 | Tempo total: 0,001905 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 200 | Escrita: 200 | Comparações: 1047 | Tempo total: 0,002054 ms.

2.2.2 Testes com 1.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 1.000 | Escrita: 1.000 | Comparações: 2.994 | Tempo total: 0,001264 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 3.000 | Escrita: 3.000 | Comparações: 9.715 | Tempo total: 0,004756 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 3.000 | Escrita: 3.000 | Comparações: 15.817 | Tempo total: 0,006468 ms.

2.2.3 Testes com 10.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 10.000 | Escrita: 10.000 | Comparações: 29.994 | Tempo total: 0,005658 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 30.000 | Escrita: 30.000 | Comparações: 94.338 | Tempo total: 0,022914 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 30.000 | Escrita: 30.000 | Comparações: 177.569 | Tempo total: 0,031495 ms.

2.2.4 Testes com 100.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 100.000 | Escrita: 100.000 | Comparações: 309.207 | Tempo total: 0,046703 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 300.000 | Escrita: 300.000 | Comparações: 906.652 | Tempo total: 0,128211 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 400.000 | Escrita: 400.000 | Comparações: 2.141.565 | Tempo total: 0,191577 ms.

2.2.5 Testes com 471.705 de Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 471.705 | Escrita: 471.705 | Comparações: 1.473.646 | Tempo total: 0,185076 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 1.886.820 | Escrita: 1.886.820 | Comparações: 4.840.037 | Tempo total: 0,550114 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 2.358.525 | Escrita: 2.358.525 | Comparações: 10.671.328 | Tempo total: 0,780387 ms.

2.3 Quick Sort Externo

O Quicksort Externo é um método de ordenação externa baseado no princípio de divisão e conquista que opera realizando acessos diretos e alternados no próprio arquivo de trabalho, sem a necessidade de fitas magnéticas temporárias. O método funciona a partir de uma função de particionamento que gerencia dois ponteiros virtuais posicionados nas extremidades do arquivo físico (limites inferior e superior) e uma área de pivôs em RAM com capacidade para até 20 registros. À medida que o arquivo é percorrido das pontas em direção ao centro, os registros lidos são comparados com os limites da memória interna, elementos menores que o limite inferior são gravados diretamente no início do arquivo, elementos maiores que o limite superior são gravados no final, e elementos que caem no intervalo intermediário entram na memória RAM, expulsando o registro mais antigo (mínimo ou máximo) para o disco de forma a balancear as partições. Esse processo de divisão física se repete de maneira recursiva para cada uma das subpartições geradas até que todo o arquivo esteja completamente ordenado no disco.

2.3.1 Testes com 100 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 224 | Escrita: 224 | Comparações: 341 | Tempo total: 0,001558 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 224 | Escrita: 224 | Comparações: 1.409 | Tempo total: 0,001571 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 224 | Escrita: 224 | Comparações: 1.391 | Tempo total: 0,001620 ms.

2.3.2 Testes com 1.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 4.917 | Escrita: 4.917 | Comparações: 8.771 | Tempo total: 0,033546 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 4.917 | Escrita: 4.917 | Comparações: 32.786 | Tempo total: 0,036886 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 4.945 | Escrita: 4.945 | Comparações: 17.235 | Tempo total: 0,033872 ms.

2.3.3 Testes com 10.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 80.462 | Escrita: 80.462 | Comparações: 150.704 | Tempo total: 0,249808 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 80.462 | Escrita: 80.462 | Comparações: 431.195 | Tempo total: 0,250166 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 81.049 | Escrita: 81.049 | Comparações: 175.236 | Tempo total: 0,260947 ms.

2.3.4 Testes com 100.000 Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 1.144.618 | Escrita: 1.144.618 | Comparações: 4.072.681 | Tempo total: 3,003539 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 1.144.618 | Escrita: 1.144.618 | Comparações: 5.436.531 | Tempo total: 3,017971 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 1.144.918 | Escrita: 1.144.918 | Comparações: 1.749.181 | Tempo total: 3,103162 ms.

2.3.5 Testes com 471.705 de Registros

- **Situação 1 (Ordenado Ascendentemente):** Leituras: 6.453.287 | Escrita: 6.453.287 | Comparações: 29.473.714 | Tempo total: 17,008171 ms.
- **Situação 2 (Ordenado Descendentemente):** Leituras: 6.453.287 | Escrita: 6.453.287 | Comparações: 29.708.079 | Tempo total: 17,040350 ms.
- **Situação 3 (Desordenado Aleatoriamente):** Leituras: 6.486.209 | Escrita: 6.486.209 | Comparações: 9.690.808 | Tempo total: 17,475090 ms.

3 Análise comparativa de desempenho entre os métodos

3.1 Intercalação Balanceada de Vários Caminhos com ordenação interna na geração de blocos

A análise de desempenho da Intercalação Balanceada de Vários Caminhos com ordenação interna evidencia a rigidez estrutural e a estabilidade característica do método frente às variações da ordem inicial dos dados. O número de transferências em disco (leituras e escritas) manteve-se rigorosamente idêntico em todas as três situações para cada respectiva volumetria de entrada, escalando linearmente conforme o número de passadas necessárias para fundir os blocos iniciais gerados a partir do limite da memória interna (20 registros). Esse comportamento simétrico comprova que o fluxo de acessos à memória externa independe do arranjo físico original das chaves. Por outro lado, a eficiência da CPU é afetada pela disposição inicial, nos cenários pré-ordenados (Situações 1 e 2), o custo computacional diminui consideravelmente, resultando em marcas de tempo de execução e em um volume de comparações menores. Em contrapartida, no cenário desordenado aleatoriamente (Situação 3), o esforço do algoritmo é intensificado, o que é refletido em um salto significativo no número de comparações de chaves, atingindo o ápice de 8.509.828 comparações e 0,796439 ms para o processamento integral da base de 471.705 registros.

3.2 Intercalação Balanceada de Vários Caminhos utilizando a de Seleção por Substituição na geração de blocos

A análise de desempenho da Intercalação Balanceada de Vários Caminhos utilizando a técnica de Seleção por Substituição na geração de blocos revela uma sensibilidade acentuada à ordem inicial das chaves. Na Situação 1 (Ordenado Ascendentemente), a ausência de congelamento no Heap-mínimo permitiu a geração de um único bloco contendo a totalidade dos registros, eliminando completamente a fase de intercalação, esse comportamento otimizado estabilizou as métricas de leitura e escrita estritamente em N acessos para todos os cenários (atingindo o mínimo absoluto de 471.705 operações no teste integral) e o menor tempo de execução. Na Situação 2 (Ordenado Descendentemente), embora a entrada invertida force o congelamento imediato do Heap na fase inicial, o padrão ordenado dos dados acelerou as passadas de fusão intermediárias, resultando em uma economia real de acessos ao disco (1.886.820 leituras e escritas) e em um menor número de comparações na CPU quando comparada ao cenário

caótico. Por fim, a Situação 3 (Desordenado Aleatoriamente) consolidou-se como o verdadeiro pior caso prático do experimento, a imprevisibilidade das notas flutuantes sobrecarregou a reestruturação do Heap, fazendo o volume de comparações explodir para 10.671.328 e exigindo o maior número de leituras e escritas (2.358.525), o que consequentemente elevou o tempo total de execução ao patamar mais alto do teste (0,780387 ms).

3.3 Quick Sort Externo

A análise de desempenho do Quicksort Externo revela uma dinâmica particular onde os acessos à mídia física exercem maior influência no tempo total de execução, enquanto o esforço de processamento na CPU apresenta um comportamento inverso ao esperado. Nas Situações 1 (Ordenado Ascendentemente) e 2 (Ordenado Descendentemente), as métricas de leitura e escrita mantiveram-se rigorosamente idênticas em quase todas as faixas amostrais (registrando exatamente 6.453.287 leituras e escritas no teste com 471.705 registros). Em contrapartida, a Situação 3 (Desordenado Aleatoriamente) exigiu uma quantidade ligeiramente maior de leituras e escritas em disco (6.486.209), o que acabou se refletindo diretamente no tempo de execução, tornando o cenário aleatório o mais lento do experimento em todas as escalas, com o ápice de 17,475090 ms. Por outro lado, o volume de comparações na CPU mostrou que os arquivos pré-estruturados sobrecarregaram o rearranjo da memória interna, fazendo as comparações saltarem para a casa dos 29 milhões nas Situações 1 e 2, enquanto o cenário aleatório demandou um esforço significativamente menor, com apenas 9.690.808 comparações na totalidade da base.

3.4 Comparação Geral

Método 1: Intercalação Balanceada de Vários Caminhos com ordenação interna na geração de blocos

Método 2: Intercalação Balanceada de Vários Caminhos utilizando a de Seleção por Substituição na geração de blocos

Método 3: Quick Sort Externo

3.4.1 100 registros

- **Ordenado Ascendentemente:**

Melhor Tempo -> Método 2 (0,000674 ms)

Menos Leituras -> Método 2 (100)

Menos Escritas -> Método 2 (100)

Menos Comparações -> Método 2 (294)

- **Ordenado Descendentemente:**

Melhor Tempo -> Método 3 (0,001571 ms)

Menos Leituras -> Método 1, 2 e 3 (200)

Menos Escritas -> Método 1, 2 e 3 (200)

Menos Comparações -> Método 1 (570)

- **Desordenado Aleatoriamente:**

Melhor Tempo -> Método 3 (0,001620 ms)

Menos Leituras -> Método 1 e 2 (200)

Menos Escritas -> Método 1 e 2 (200)

Menos Comparações -> Método 1 (877)

3.4.2 1.000 registros

- **Ordenado Ascendentemente:**

Melhor Tempo -> Método 2 (0,001264 ms)

Menos Leituras -> Método 2 (1.000)

Menos Escritas -> Método 2 (1.000)

Menos Comparações -> Método 2 (2.994)

- **Ordenado Descendentemente:**

Melhor Tempo -> Método 2 (0,004756 ms)

Menos Leituras -> Método 1 e 2 (3.000)

Menos Escritas -> Método 1 e 2 (3.000)

Menos Comparações -> Método 1 (6.932)

- **Desordenado Aleatoriamente:**

Melhor Tempo -> Método 2 (0,006468 ms)

Menos Leituras -> Método 1 e 2 (3.000)

Menos Escritas -> Método 1 e 2 (3.000)

Menos Comparações -> Método 1 (12.654)

3.4.3 10.000 registros

- **Ordenado Ascendentemente:**

Melhor Tempo -> Método 2 (0,005658 ms)

Menos Leituras -> Método 2 (10.000)

Menos Escritas -> Método 2 (10.000)

Menos Comparações -> Método 2 (29.994)

- **Ordenado Descendentemente:**

Melhor Tempo -> Método 2 (0,022914 ms)

Menos Leituras -> Método 1 e 2 (30.000)

Menos Escritas -> Método 1 e 2 (30.000)
Menos Comparações -> Método 1 (80.382)

- **Desordenado Aleatoriamente:**

Melhor Tempo -> Método 2 (0,031495 ms)
Menos Leituras -> Método 1 e 2 (30.000)
Menos Escritas -> Método 1 e 2 (30.000)
Menos Comparações -> Método 1 (152.298)

3.4.4 100.000 registros

- **Ordenado Ascendentemente:**

Melhor Tempo -> Método 2 (0,046703 ms)
Menos Leituras -> Método 2 (100.000)
Menos Escritas -> Método 2 (100.000)
Menos Comparações -> Método 2 (309.207)

- **Ordenado Descendentemente:**

Melhor Tempo -> Método 2 (0,128211 ms)
Menos Leituras -> Método 2 (300.000)
Menos Escritas -> Método 2 (300.000)
Menos Comparações -> Método 2 (906.652)

- **Desordenado Aleatoriamente:**

Melhor Tempo -> Método 1 (0,167314 ms)
Menos Leituras -> Método 1 e 2 (400.000)
Menos Escritas -> Método 1 e 2 (400.000)
Menos Comparações -> Método 1 (1.687.249)

3.4.5 471.705 registros

- **Ordenado Ascendentemente:**

Melhor Tempo -> Método 2 (0,185076 ms)
Menos Leituras -> Método 2 (471.705)
Menos Escritas -> Método 2 (471.705)
Menos Comparações -> Método 2 (1.473.646)

- **Ordenado Descendentemente:**

Melhor Tempo -> Método 2 (0,550114 ms)

Menos Leituras -> Método 2 (1.886.820)

Menos Escritas -> Método 2 (1.886.820)

Menos Comparações -> Método 2 (4.840.037)

- **Desordenado Aleatoriamente:**

Melhor Tempo -> Método 2 (0,780387 ms)

Menos Leituras -> Método 1 e 2 (2.358.525)

Menos Escritas -> Método 1 e 2 (2.358.525)

Menos Comparações -> Método 1 (8.509.828)

4 Principais dificuldades de implementação dos métodos

4.1 Intercalação Balanceada de Vários Caminhos com ordenação interna na geração de blocos

A implementação da Intercalação Balanceada de Vários Caminhos utilizando ordenação interna para a geração de blocos impôs desafios lógicos complexos, concentrados principalmente no gerenciamento rigoroso dos fluxos de arquivos e na sincronização de estruturas de dados na memória RAM. A principal dificuldade esteve no controle das fitas magnéticas virtuais, o que exigiu uma lógica minuciosa para alternar dinamicamente quais arquivos estão em modo de leitura ou escrita a cada passada, tratando corretamente a abertura, fechamento e permissões no disco para evitar erros. Somado a isso, o algoritmo demandou um rastreamento exato dos limites lógicos de cada bloco durante a fase de fusão para impedir que o ponteiro de leitura invada um bloco vizinho na mesma fita, além de exigir uma estrutura de prioridades (como um Heap-minímo) perfeitamente sincronizada para associar cada registro à sua fita de origem.

4.2 Intercalação Balanceada de Vários Caminhos utilizando a de Seleção por Substituição na geração de blocos

A implementação da Intercalação Balanceada de Vários Caminhos utilizando a técnica de Seleção por Substituição para a geração de blocos trouxe uma dificuldade muito maior porque exigiu controlar o que acontece na memória RAM ao mesmo tempo em que os dados são gravados no disco. O maior desafio esteve em gerenciar o Heap-minímo para isolar os registros "congelados" (aqueles que chegam do arquivo com uma chave menor do que o último dado gravado no bloco atual) para que eles não entrem na partição que está sendo criada. Essa mecânica exigiu uma lógica complicada para reorganizar o Heap, onde o algoritmo precisa diminuir o tamanho útil da estrutura e perceber o momento exato em que todos os elementos foram congelados, é esse evento que avisa que o bloco atual nas fitas deve ser fechado, todos os registros devem ser descongelados e um novo bloco deve começar. Além disso, coordenar

essa troca de dados entre a memória interna e os arquivos de saída gerou muitos erros de ponteiros e falhas de segmentação, exigindo uma lógica perfeito para garantir que nenhum registro seja esquecido ou gravado no lugar errado.

4.3 Quick Sort Externo

A implementação do Quicksort Externo trouxe uma dificuldade central que é o controle preciso dos ponteiros de leitura e escrita diretamente no próprio arquivo de trabalho, dispensando o uso de fitas temporárias. O maior desafio esteve em gerenciar de forma sincronizada dois ponteiros virtuais que começam nas extremidades do arquivo físico e andam em direção ao centro, exigindo o uso constante de funções de posicionamento como o `fseek` e `ftell`. Coordenar esses acessos alternados e diretos no disco com uma área de pivôs na memória RAM (que guarda até 20 registros) exigiu uma lógica complexa para decidir qual registro deve ser lido, comparado e reescrito sem que um ponteiro atropеле o outro. Além disso, a implementação da recursão para ordenar as subpartições geradas em disco exigiu um controle rígido dos limites de bytes de cada pedaço do arquivo, pois qualquer erro no cálculo dos deslocamentos (offsets) pode sobrescrever dados válidos, travar o programa em loops infinitos ou gerar falhas de segmentação.